RELATÓRIO DE AVALIAÇÃO DE VULNERABILIDADES

Teste de Segurança Pré-Produção — Red Team

Aplicação alvo: ElectroMarket
URL: https://mercado-electro.vercel.app/
Plataforma: Vercel (hosting) + Supabase (backend/DB) — arquitetura client-side
Data do teste: 09/11/2026
Tipo: Teste externo não-autenticado, não-destrutivo (recon + análise de configuração)
Escopo: Avaliação pré-produção autorizada pelo proprietário da aplicação
Classificação geral de risco: CRÍTICO — não recomendado para produção no estado atual

Confidencial. Documento gerado para validação de segurança antes de deploy.

1. Sumário Executivo

Foi realizada uma avaliação de segurança externa, não-autenticada e não-destrutiva da aplicação ElectroMarket antes de sua promoção para produção. A avaliação identificou uma vulnerabilidade CRÍTICA que, isoladamente, compromete integralmente a confidencialidade, integridade e disponibilidade dos dados: a chave de maior privilégio do backend (Supabase service_role) está publicamente exposta ao cliente.

Enquanto essa falha existir, qualquer pessoa na internet possui acesso administrativo total ao banco de dados. A recomendação é NÃO promover a aplicação para produção até a remediação dos itens Crítico e Altos, e a reavaliação subsequente.

1.1. Quadro de achados

ID	Vulnerabilidade	Severidade	CVSS
RT-01	Chave service_role do Supabase exposta no cliente	CRÍTICA	10.0
RT-02	Endpoint /api/config sem autenticação expõe segredos de configuração	ALTA	7.5
RT-03	Headers de segurança HTTP ausentes	MÉDIA	6.1
RT-04	CORS permissivo (Access-Control-Allow-Origin: *)	MÉDIA	5.3
RT-05	Lógica de administração e vendedor entregue ao cliente	MÉDIA	5.3
RT-06	Ausência de validação e moderação de conteúdo dos anúncios	MÉDIA	5.4
RT-07	Dependências de terceiros via CDN sem fixação de versão nem SRI	BAIXA	3.7

Distribuição: 1 Crítica · 1 Alta · 4 Médias · 1 Baixa

1.2. Metodologia

Reconhecimento passivo e ativo não-destrutivo seguindo OWASP Testing Guide v4.2 e OWASP API Security Top 10. As interações limitaram-se a requisições de leitura equivalentes às que o próprio navegador de qualquer visitante já realiza. Nenhum dado real de produção foi exfiltrado, alterado ou excluído; as provas de conceito usam apenas metadados (status HTTP, contagem de registros, nomes de colunas e claims públicos de JWT).

2. Achados Detalhados

RT-01 — Chave service_role do Supabase exposta no cliente

SEVERIDADE: CRÍTICA

CVSS v3.1: 10.0 (AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)
CWE: CWE-798 / CWE-200 (Hard-coded Credentials / Information Exposure)
OWASP: A07:2021 Identification & Authentication Failures / A05 Security Misconfiguration
Localização: /api/config e JavaScript client-side (window.supabase client)

Descrição

A aplicação entrega, para qualquer visitante anônimo, a SUPABASE_URL e a SUPABASE_KEY através do endpoint público /api/config. A key fornecida é um JWT cujo claim "role" é service_role — a chave de maior privilégio do Supabase, que IGNORA completamente as políticas de Row Level Security (RLS). Com ela, qualquer pessoa na internet tem acesso administrativo irrestrito (leitura, escrita, atualização e exclusão) a todas as tabelas do banco via a API REST do Supabase.

Evidência

- `GET https://mercado-electro.vercel.app/api/config → 200 OK`
- `Resposta contém: SUPABASE_URL, SUPABASE_KEY (JWT, 219 chars), GOOGLE_CLIENT_ID (vazio)`
- `Decodificação do claim do JWT (somente o payload, sem expor a chave): role = "service_role", ref = "pjisiqvaulgoikaitmaj", exp = 2092638980 (≈ ano 2036 — chave de longa duração)`
- `PoC não-destrutivo (somente leitura, com contagem exata via header Range): tabela products = 35 registros; tabela users = 13 registros; tabela orders = 7 registros — todas acessíveis com a chave exposta.`

Impacto no negócio

Comprometimento total do banco de dados. Um atacante pode: ler todos os dados de usuários (PII, e-mails, credenciais se armazenadas), ler e adulterar pedidos (orders), alterar preços de produtos, inserir conteúdo malicioso, e EXCLUIR qualquer dado. Em produção isso representa violação de LGPD (Art. 46 — falha em medidas de segurança), risco de vazamento de dados pessoais reportável à ANPD, fraude e indisponibilidade total do serviço.

Remediação

- IMEDIATO: rotacionar a service_role key no painel Supabase (Settings → API). Considerar a chave atual permanentemente comprometida.
- NUNCA servir a service_role ao cliente nem pelo /api/config. O frontend deve usar exclusivamente a anon key (public).
- Mover toda operação privilegiada (admin, seller, escrita de orders) para funções server-side (Vercel Serverless/Edge Functions) que guardam a service_role como variável de ambiente secreta (Environment Variables, nunca em /api/config público).
- Habilitar RLS em todas as tabelas com políticas explícitas por papel. Mesmo com a anon key, sem RLS o banco permanece aberto.
- Auditar logs do Supabase para identificar acessos indevidos já ocorridos com a chave exposta.

RT-02 — Endpoint /api/config sem autenticação expõe segredos de configuração

SEVERIDADE: ALTA

CVSS v3.1: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
CWE: CWE-200 Exposure of Sensitive Information

OWASP: A05:2021 Security Misconfiguration

Localização: GET /api/config

Descrição

O endpoint retorna um objeto JSON de configuração sem qualquer controle de acesso. Além da service_role (RT-01), o padrão de expor configuração sensível por um endpoint público é uma falha estrutural: qualquer segredo colocado ali vaza para a internet inteira.

Evidência

- `GET /api/config → 200 OK, JSON com chaves SUPABASE_URL, SUPABASE_KEY, GOOGLE_CLIENT_ID, sem header de autenticação exigido.`

Impacto no negócio

Vazamento de qualquer credencial/segredo de integração presente na configuração. Facilita automação de ataques (a URL do projeto e a chave são obtidas num único GET).

Remediação

- Eliminar o endpoint /api/config como fonte de segredos. O cliente só deve receber valores realmente públicos (ex.: anon key), idealmente embutidos no build, não por API.
- Segredos (service_role, client secrets) ficam apenas em variáveis de ambiente server-side.
- Se precisar de config dinâmica no cliente, exponha somente valores não-sensíveis e explicitamente allow-listed.

RT-03 — Headers de segurança HTTP ausentes

SEVERIDADE: MÉDIA

CVSS v3.1: 6.1 (AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)

CWE: CWE-693 Protection Mechanism Failure

OWASP: A05:2021 Security Misconfiguration

Localização: Resposta HTTP de https://mercado-electro.vercel.app/

Descrição

A resposta não define cabeçalhos de segurança essenciais, deixando a aplicação exposta a clickjacking, MIME-sniffing, injeção de scripts e vazamento de referer.

Evidência

- `Content-Security-Policy: AUSENTE`
- `X-Frame-Options: AUSENTE (risco de clickjacking)`
- `X-Content-Type-Options: AUSENTE (risco de MIME-sniffing)`
- `Referrer-Policy: AUSENTE`
- `Permissions-Policy: AUSENTE`
- `Strict-Transport-Security: PRESENTE (único controle correto)`

Impacto no negócio

Sem CSP, qualquer injeção de HTML/JS (ver RT-06) vira XSS executável. Sem X-Frame-Options/frame-ancestors, a página pode ser embutida em iframe para clickjacking. Sem X-Content-Type-Options, respostas podem ser interpretadas como tipos perigosos.

Remediação

- Definir CSP restritiva. Atenção: hoje o app depende de várias CDNs (jsdelivr, unpkg, fonts.googleapis), que precisarão ser allow-listed — ou melhor, auto-hospedadas.

- X-Content-Type-Options: nosniff; X-Frame-Options: DENY (ou frame-ancestors 'none' na CSP); Referrer-Policy: strict-origin-when-cross-origin; Permissions-Policy desabilitando APIs não usadas.
- No Vercel, configurar via headers no vercel.json ou next.config.

RT-04 — CORS permissivo (Access-Control-Allow-Origin: *)

SEVERIDADE: MÉDIA

CVSS v3.1: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CWE: CWE-942 Permissive Cross-domain Policy
OWASP: A05:2021 Security Misconfiguration
Localização: Resposta HTTP (header Access-Control-Allow-Origin)

Descrição

A aplicação responde com Access-Control-Allow-Origin: *. Não há Access-Control-Allow-Credentials, o que limita a gravidade, mas a política aberta permite que qualquer site consuma as respostas da origem.

Evidência

- `Access-Control-Allow-Origin: * ; Access-Control-Allow-Credentials: AUSENTE.`

Impacto no negócio

Qualquer origem pode ler respostas públicas. Combinado com endpoints que devolvam dados sensíveis, amplia a superfície de exfiltração via browser de vítimas.

Remediação

- Restringir ACAO a origens confiáveis explícitas (o próprio domínio de produção).
- Nunca combinar ACAO:* com credenciais.

RT-05 — Lógica de administração e vendedor entregue ao cliente

SEVERIDADE: MÉDIA

CVSS v3.1: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N)
CWE: CWE-602 Client-Side Enforcement of Server-Side Security
OWASP: A01:2021 Broken Access Control
Localização: /js/admin.js, /js/seller.js, /js/cliente.js (servidos a usuários anônimos)

Descrição

Arquivos JavaScript de admin e seller são servidos publicamente a qualquer visitante. Em uma arquitetura client-side + Supabase, isto normalmente significa que as regras de autorização (quem pode o quê) estão no front-end — e podem ser contornadas editando o JS ou chamando a API diretamente. Com a service_role exposta (RT-01), o controle de acesso é inexistente; mesmo após corrigir RT-01, a autorização precisa ser no servidor/RLS.

Evidência

- `GET /js/admin.js e /js/seller.js retornam 200/304 para sessão anônima (observado no tráfego de rede do carregamento da home).`

Impacto no negócio

Escalonamento de privilégio: um usuário comum pode acionar funções administrativas ou de vendedor chamando as mesmas operações de banco diretamente, se a autorização não for imposta no backend/RLS.

Remediação

- Impor autorização no servidor (RLS por papel + funções server-side). O front-end nunca é fonte de verdade de permissão.
- Não expor lógica administrativa sensível em bundles públicos; segregar o painel admin atrás de autenticação e, idealmente, em rota/app separada.

RT-06 — Ausência de validação e moderação de conteúdo dos anúncios

SEVERIDADE: MÉDIA

CVSS v3.1: 5.4 (AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N)
CWE: CWE-79 / CWE-20 (Improper Input Validation / potencial Stored XSS)
OWASP: A03:2021 Injection
Localização: Fluxo de criação de anúncios (dados renderizados na home)

Descrição

A vitrine exibe anúncios de teste com conteúdo claramente abusivo/ilícito e dados malformados: títulos como "AK 47", "Erva do Fabio", "cigarro eletrico e esquilo", e preços quebrados ("R$ 1,.00", "R$ 8.999,.90"). Isso evidencia (a) falta de moderação/validação de conteúdo e (b) falha de validação de tipos/valores. Se o título/descrição são renderizados sem sanitização e sem CSP (RT-03), há risco concreto de Stored XSS persistente a partir de um anúncio.

Evidência

- `Títulos de anúncios na home: "AK 47", "Erva do Fabio", "cigarro eletrico e esquilo", "Maquina de Shopping" (100% OFF / R$ 0).`
- `Preços inconsistentes: "R$ 1,.00", "R$ 3.499,.00", "R$ 8.999,.90" (formatação/validação numérica quebrada).`
- `Produto com desconto de 100% / valor GRÁTIS — falha de regra de negócio (valores ≤ 0).`

Impacto no negócio

Risco legal e reputacional (venda aparente de armas/drogas), fraude por preços inválidos, e — sem sanitização + sem CSP — execução de JavaScript arbitrário no navegador de outros usuários (roubo de sessão, defacement).

Remediação

- Validar server-side todos os campos (comprimento, tipo, faixa de preço > 0, categoria permitida).
- Sanitizar/escapar toda saída renderizada (textContent em vez de innerHTML; biblioteca de sanitização se HTML for permitido).
- Implementar moderação de conteúdo (lista de termos proibidos, fila de revisão, denúncia).
- Aplicar CSP (RT-03) como defesa em profundidade contra XSS.

RT-07 — Dependências de terceiros via CDN sem fixação de versão nem SRI

SEVERIDADE: BAIXA

CVSS v3.1: 3.7 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N)
CWE: CWE-829 Inclusion of Functionality from Untrusted Control Sphere
OWASP: A08:2021 Software & Data Integrity Failures
Localização: <script>/<link> para jsdelivr, unpkg, fonts.googleapis

Descrição

A aplicação carrega bibliotecas de CDNs públicas, incluindo versões não fixadas (ex.: @rive-app/canvas@latest via unpkg, @supabase/supabase-js@2) e sem atributo Subresource Integrity (SRI). Um comprometimento da CDN ou do pacote injetaria código arbitrário na aplicação.

Evidência

- `https://unpkg.com/@rive-app/canvas@latest (versão flutuante)`
- `https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2 (minor/patch flutuante)`
- `bootstrap, chart.js, bootstrap-icons via jsdelivr — sem integrity=.`

Impacto no negócio

Supply-chain: injeção de código client-side que, dado o acesso Supabase do front, poderia manipular dados e sessões.

Remediação

- Fixar versões exatas e adicionar integrity (SRI) + crossorigin em todos os <script>/<link> de CDN.
- Preferir auto-hospedar as dependências no próprio deploy (facilita também a CSP do RT-03).

---

3. Plano de Ação Priorizado

Prioridade	Ação	Prazo	Criticidade
P0 — Agora	Rotacionar a service_role key e remover qualquer segredo do /api/config (RT-01, RT-02)	Antes de qualquer deploy	Bloqueante
P1 — 24–48h	Habilitar RLS em todas as tabelas + mover operações privilegiadas para backend (RT-01, RT-05)	Antes de produção	Bloqueante
P1 — 24–48h	Validação/sanitização de inputs e moderação de anúncios (RT-06)	Antes de produção	Alto
P2 — 1 semana	Security headers (CSP, XFO, nosniff, Referrer/Permissions-Policy) e CORS restrito (RT-03, RT-04)	Pré ou logo após go-live	Médio
P3 — Backlog	Fixar versões + SRI / auto-hospedar dependências (RT-07)	Melhoria contínua	Baixo

3.1. Veredito

A aplicação NÃO deve ser promovida para produção enquanto RT-01 e RT-02 não forem corrigidos e validados. Após a remediação dos itens P0/P1, recomenda-se um reteste focado em RLS (tentativas de leitura/escrita com a anon key) e em controle de acesso por papel.

— Fim do relatório —

Gerado em avaliação autorizada. As provas de conceito são não-destrutivas e não envolveram exfiltração de dados pessoais.